

算分第二次作业. 杨象铭

第二章

2.7

解: (1) 先用 $O(n \log n)$ 时间完成排序,

然后用 $O(n)$ 时间遍历一遍这个有序列, 看是否有连续 $\lfloor \frac{n}{2} \rfloor + 1$ 个数都相同即可

(2) 首先, 注意到若存在主元素, 那么这个主元素一定是中位数.

那么又由 Select 算法, 一般性选第 k 大元素可在 $O(n)$ 内解决

于是可以 $O(n)$ 找到中位数, 接下来再 $O(n)$ 遍历比较, 看是否有 $\lfloor \frac{n}{2} \rfloor$ 及以上元素与中位数相同即可

(3)

① 最直接的想法是 $O(n^2)$ 的 2 层循环遍历, 对每个元素看是否满足主元素

② 设计 Delete 策略: $(O(n))$

Delete(A):

if $|A| \leq 1$ 结束

else 两两分组比较.

相同留下一个, 不同全弃.

return A 并回到 if 判断.

首先, 可以注意到 $|A|$ 为偶数时, 删除操作保持性质:

<1> 删后主元素个数减少 $\leq$ 非主 ~ 减少.

<2> $|A|$ 删后至多为原来一半.

而对于 $|A|$ 为奇数时, 若主元素轮空, 性质 <1> 不能保持. 因为奇数时, 要对轮空元素做额外 $O(n)$ 判断, 是主元素则结束, 不是则先剔除, 回到偶数情况

那么综上, 性质1表明如果存在主元素, 一定会留到最后.
所以最后只需对剩下元素做一次 $O(n)$ 遍历与验证即可. $\square$.

而对于复杂度,

由性质2, 删除操作次数为 $O(\log n)$

递归:
$$\begin{cases} W(n) = W(\frac{n}{2}) + O(n) \rightarrow \text{奇数特判} \\ W(1) = 0 \end{cases}$$

$n^{\log 2} = n^0$ 主定理 $\checkmark$

$\Rightarrow W(n) = O(n)$

2.12 解:

先对 B (较小) 排序, 复杂度 $O(m \log m)$

然后可以在 A 中遍历, B 中二分查找, 伪代码如下:

输入: A, B

输出: $C = A \cap B$

$C = \emptyset$ (初始化)

$\text{Sort}(B)$

for i in A : (n 个)

$j = \text{BinarySearch}(B, i)$

if $j > 0$

$C = C \cup j$

return C

2.15.

解: (1) $A = \text{Findmax}: O(n)$

$$\therefore W_A = O(n) \cdot i = O(n^{\frac{3}{2}}) \\ O(i \cdot n)$$

$B = \text{排序}: O(n \log n)$

(2) 考虑运用 select 方法. 设原数列为 S

$\text{select}(S, n-i)$ 即找到 S 中第 i 大的元素. 复杂度为 $O(n)$
(记为 k)

然后遍历 S 中元素, 与 k 依此比较. 所有更大的元素; 共 i 个,
记为 S' (复杂度 $O(n)$)

最后由大到小排列 S' 即可. 复杂度 $O(i \log i) \leq O(n^{\frac{1}{2}} \cdot \log(n^{\frac{1}{2}}))$

综上, 我们找到复杂度为 $O(n)$ 的更优方法

2.18

解: 与上 2.15 类似, 首先我们把 $x_i, \sqrt{x_i y_i} = S_i$ 算出来.

下面只需研究序列 S .

还是先 select 第 k 小, $O(n)$, 记为 S_k

遍历 S (n 个), 把比 S_k 小的筛选出来. $O(n)$

最后对 $\lfloor \sqrt{n} \rfloor$ 个元素从大到小排序, $O(\lfloor \sqrt{n} \rfloor \log(\lfloor \sqrt{n} \rfloor))$

伪码: $C = \phi$
 $S_k = \text{select}(S, i) \rightarrow$ 设为取第 i 小元素

for i in S :
 if ($i < S_k$):

$C = i \cup C$

$\text{sort}(C) \rightarrow$ 从大到小. (less)

按序输出 C 中元素即可

3.3

解:

边界: $\sum l_k \leq D$.

目标: $\sum v_k \max$.

设 $F_k(y)$ 表示在前 k 种货柜中选择, 带来的最大收益
长度不超过 y .

于是有:

边界条件: $F_0(y) = 0$

$F_k(0) = 0$

→ 当 $y \geq l_k$, 否则只能是 F_k .

递推: $F_k(y) = \max \{F_{k-1}(y), F_{k-1}(y - l_k) + v_k\}$

伪码:

输入: $L[1 \sim n], V[1 \sim n], D(\text{长度})$

输出: $F_n(y) \Rightarrow F_n(D)$ 特别地

for $y = 1$ to D : $O(D)$

if $y \geq L[1]$:

$F_1(y) = V[1]$

for $k = 2$ to n :
for $y = 1$ to D : $O(nD)$

$F_k(y) = F_{k-1}(y)$

if $y \geq L[k]$ 且 $F_{k-1}(y - L[k]) + V[k] > F_{k-1}(y)$:

$F_k(y) = F_{k-1}(y - L[k]) + V[k]$.

return $F_n(D)$

复杂度为 $O(nD)$

3.8

首先判断 N 为奇直接输出 No .

N 为偶, 设 $B = \frac{N}{2}$.

设划为 $A_1, \dots, A_2$, 不妨设 A_1 中元素和 $\leq A_2$.

那么 $\sum_{a \in A_1} a \leq B$.

那么我们即要求从 A 中选若干元素为 A_0 .

$$\sum_{a' \in A_0} a' \leq B, \text{ 且 } \max \sum_{a' \in A_0} a'$$

化为类似背包的组合优化:

考虑 $F_k(y)$ 表示考虑前 k 个数, 总和不超过 y 时, 和最大值.

那么递推 $F_k(y) = \max \{ F_{k-1}(y), F_{k-1}(y - a_k) + a_k \}$.

边界条件 $F_1(y) = \begin{cases} a_1, & y \geq a_1 \\ 0, & y < a_1 \end{cases}$

那么复杂度为 $O(n \cdot B)$ 的两层循环后可解得 $F_n(B)$.

若 $F_n(B) = B$, 则 yes; 否则 No.

3.10.

解: 多一个限制条件.

此题即找 $\max \sum_{i \in [1, n]} V_i$, 且 $\sum w_i < W$ $\sum c_i \leq V$

那么我们设 $f(i, j, k)$ 为考虑前 i 个物品, 体积不超过 k , 重量不超过 j 的最大价值.

初始条件 $f(1, j, k) = \begin{cases} V_1, & j \geq w_1 \text{ 且 } k \geq c_1 \\ 0, & j < w_1 \text{ 或 } k < c_1 \end{cases}$

递推: $f(i, j, k) = \max \{ f(i-1, j, k), f(i-1, j - w_i, k - c_i) + V_i \}$.

在代码中, 表示为三层循环, 故复杂度为 $O(n \cdot W \cdot V)$

3.17

解

数字和最小

设到点 a_{ij} 的“最短路径”记为 $f(i, j)$.

$$\text{那么 } f(1, 1) = a_{11}$$

考虑对每一行 i , 前 j 个点的数字和最小的路径为 $g(i, j)$.

$$g(i, j) = \min \{ g(i, j-1), g(i-1, j-1) + a_{ij}, g(i-1, j) + a_{ij} \}.$$

(当然如果第 $i-1$ 行没有第 j 个就不考虑第三种可能.)
与背包问题中 " $y - w_i < 0$ " 情况一样

再看初始条件: $g(1, 1) = a_{11}$.

$$g(i, 1) = \sum_{k=1}^i a_{k1}$$

(在循环内)

之后只要对 i 循环, 内循环 j , 递归更新即可, 时间复杂度为 $O(n^2)$